

Objetivos da aula:

- Compreender o conceito e o funcionamento de triggers no PostgreSQL;
- Criar e utilizar triggers BEFORE, AFTER e INSTEAD OF;
- Utilizar corretamente as pseudo-variáveis NEW e OLD;
- Entender o papel dos cursores e seu uso em lógica procedural;
- Avaliar quando o uso de triggers é adequado, considerando manutenção e desempenho.

Índice:

- i. Introdução a Triggers (Gatilhos)
- ii. Triggers na prática
- iii. Triggers Avançados: INSTEAD OF e TRUNCATE

i. Introdução a Triggers (Gatilhos)

1.1. Introdução

Em sistemas de banco de dados relacionais, muitas regras de negócio e validações precisam ser executadas **automaticamente** sempre que determinados eventos ocorrem, como inserções, atualizações ou exclusões de dados.

Para atender a essa necessidade, os SGBDs oferecem o mecanismo de triggers (gatilhos), que permitem a execução automática de código no momento em que um evento específico acontece em uma tabela.

No PostgreSQL, triggers são amplamente utilizadas para:

- validação automática de dados;
- auditoria e histórico de alterações;
- aplicação de regras de negócio;
- sincronização entre tabelas;
- prevenção de operações inválidas.

Um trigger é um objeto do banco de dados que:

- está associado a uma tabela (ou view);
- é disparado automaticamente por um evento;
- executa uma função previamente definida;
- não é chamado diretamente pelo usuário.

Definição conceitual:

Um trigger é um mecanismo que executa automaticamente uma função quando ocorre um evento específico em uma tabela.

1.2. Eventos que podem disparar um trigger

No PostgreSQL, os principais eventos que podem acionar um trigger são:

- INSERT
- UPDATE
- DELETE
- TRUNCATE

Esses eventos correspondem às principais operações de modificação de dados no banco.

1.3. Momento de execução do trigger

Um trigger pode ser executado em dois momentos distintos:

1.3.1 BEFORE (antes do evento)

- Executado **antes** da operação (INSERT, UPDATE, DELETE);
- Permite validar ou modificar os dados antes que sejam gravados;
- Pode impedir a operação lançando uma exceção.

Exemplo de uso:

- impedir inserção de valores inválidos;
- ajustar automaticamente valores de colunas.

1.3.2 AFTER (após o evento)

- Executado **após** a operação ser concluída;
- Ideal para auditoria, logs e ações secundárias;
- Não altera o resultado da operação original.

Exemplo de uso:

- registrar histórico de alterações;
- atualizar tabelas auxiliares.

1.4. Relacionamento entre trigger e função

No PostgreSQL, um trigger não contém código diretamente. Ele sempre executa uma função, chamada de **função de trigger**.

Essa função:

- é escrita normalmente em PL/pgSQL;
- possui assinatura específica;
- retorna o tipo especial TRIGGER.

Conceitualmente:

Trigger → define **quando**

Função → define **o que será executado**

1.5. Estrutura geral de um trigger

1.5.1 Estrutura conceitual da função de trigger

```
CREATE FUNCTION nome_funcao_trigger()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- lógica do trigger  
    RETURN NEW;  
END;  
$$;
```

1.5.2 Estrutura conceitual do trigger

```
CREATE TRIGGER nome_trigger  
BEFORE INSERT ON tabela  
FOR EACH ROW  
EXECUTE FUNCTION nome_funcao_trigger();
```

Esses dois elementos **sempre trabalham em conjunto**.

1.6. Considerações importantes sobre triggers

Alguns pontos fundamentais que devem ser compreendidos desde o início:

- triggers são executados **automaticamente**, sem ação do usuário;
- erros em triggers podem impedir operações no banco;
- uso excessivo pode dificultar manutenção e depuração;
- regras críticas devem ser bem documentadas;
- triggers devem ser usados com critério.

1.7. Quando usar triggers

Triggers são indicados quando:

- a regra deve ser aplicada **independentemente da aplicação cliente**;
- é necessário garantir consistência no nível do banco;
- ações devem ocorrer automaticamente após eventos;
- múltiplas aplicações acessam o mesmo banco.

Não são recomendados para:

- lógica excessivamente complexa;

- regras que mudam com frequência;
- substituição completa da lógica da aplicação.

ii. Triggers na prática

2.1. Criação de uma função de trigger

Como visto no capítulo anterior, **todo trigger executa uma função**.

Essa função deve obrigatoriamente:

- retornar o tipo especial TRIGGER;
- ser escrita em PL/pgSQL.

Estrutura básica

```
CREATE FUNCTION nome_funcao_trigger()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- lógica do trigger  
    RETURN NEW; -- NEW ou OLD  
END;  
$$;
```

O valor retornado depende do tipo e do momento do trigger:

- NEW → para INSERT e UPDATE;
- OLD → para DELETE.

2.2. Uso de NEW e OLD

◆ NEW

- representa a **nova linha** que será inserida ou atualizada;
- disponível em triggers INSERT e UPDATE.

Exemplo de acesso:

```
NEW.nome  
NEW.nascimento
```

OLD

- representa a **linha antiga**, antes da modificação;
- disponível em triggers UPDATE e DELETE.

Exemplo:

```
OLD.nome
```

OLD.nascimento

Essas pseudo-variáveis permitem acessar os dados da linha diretamente dentro do trigger.

2.3. Exemplo 1 — Trigger de Validação (BEFORE INSERT)

Cenário: Impedir a inserção de uma pessoa com data de nascimento no futuro.

Função de trigger

```
CREATE OR REPLACE FUNCTION validar_nascimento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.nascimento > CURRENT_DATE THEN
        RAISE EXCEPTION 'Data de nascimento inválida';
    END IF;

    RETURN NEW;
END;
$$;
```

Trigger associado

```
CREATE OR REPLACE TRIGGER trg_validar_nascimento
BEFORE INSERT ON pessoa
FOR EACH ROW
EXECUTE FUNCTION validar_nascimento();
```

Uso do trigger

```
INSERT INTO pessoa (id_pessoa, nome, nascimento)
VALUES (100001, 'Ana Novo', '2030-01-01');
```

Reposta:

```
ERROR: Data de nascimento inválida
CONTEXT: função PL/pgSQL validar_nascimento() linha 4 em RAISE
SQL state: P0001
```

O trigger `trg_validar_nascimento` foi definido apenas para a operação de INSERT na tabela `pessoa`, o comando a seguir será executado sem problemas:

```
UPDATE pessoa
SET nascimento = '2030-01-01'
where id_pessoa = 1;
```

2.4. Exemplo 2 — Trigger de Auditoria (AFTER UPDATE)

Cenário: Registrar alterações de nome em uma tabela de auditoria.

Tabela de auditoria

```
DROP TABLE IF EXISTS pessoa_auditoria;
CREATE TABLE pessoa_auditoria (
    id_pessoa INTEGER,
    nome_antigo VARCHAR(100),
    nome_novo VARCHAR(100),
    data_alteracao TIMESTAMP
);
```

Função de trigger

```
CREATE OR REPLACE FUNCTION auditar_nome()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO pessoa_auditoria
    VALUES (
        OLD.id_pessoa,
        OLD.nome,
        NEW.nome,
        CURRENT_TIMESTAMP
    );

    RETURN NEW;
END;
$$;
```

Trigger associado

```
CREATE OR REPLACE TRIGGER trg_auditar_nome
AFTER UPDATE OF nome ON pessoa
FOR EACH ROW
EXECUTE FUNCTION auditar_nome();
```

Uso do trigger

```
UPDATE pessoa
SET nome = 'Ana Novo'
where id_pessoa = 1;
```

Inspeção do resultado:

```
SELECT *
FROM pessoa_auditoria;
```

<code>id_pessoa</code> integer	<code>nome_antigo</code> character varying (100)	<code>nome_novo</code> character varying (100)	<code>data_alteracao</code> timestamp without time zone
1	Elaine Oliveira	Ana Novo	2026-01-31 15:41:26.799146

2.5. Introdução aos Cursores (Conceito Fundamental)

O que é um cursor?

Um cursor é um mecanismo que permite **percorrer linha a linha o resultado de uma consulta**.

Conceitualmente:

Um cursor é um ponteiro para um conjunto de resultados, permitindo processar uma linha por vez.

Cursores são úteis quando:

- o processamento precisa ser sequencial;
- não é possível (ou não é desejável) tratar todas as linhas de uma vez;
- há necessidade de lógica procedural por linha.

Observação didática importante

Em PostgreSQL (e em PL/pgSQL):

- a maioria dos laços FOR ... IN SELECT usa cursores implicitamente;
- você já usa cursores, mesmo sem declará-los explicitamente.

2.6. Cursor implícito (forma mais comum)

Exemplo:

```
DO $$
DECLARE
    registro RECORD;
BEGIN
    FOR registro IN
        SELECT id_pessoa, nome FROM pessoa LIMIT 5
    LOOP
        RAISE NOTICE 'Pessoa: %', registro.nome;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Nesse caso:

- o PostgreSQL cria e gerencia o cursor automaticamente;
- o desenvolvedor não precisa abrir ou fechar o cursor;
- essa é a **forma recomendada** na maioria dos casos.

2.7. Cursor explícito (forma detalhada)

Quando usar cursor explícito?

- quando se deseja mais controle;
- em lógica mais complexa.

Exemplo de cursor explícito

```
DO $$
DECLARE
    -- 1. declaração do cursor
    cur_pessoa CURSOR FOR
        SELECT id_pessoa, nome FROM pessoa LIMIT 5;
    reg RECORD;
BEGIN
    OPEN cur_pessoa;      -- 2. abertura

    LOOP
        -- 3. leitura
        FETCH cur_pessoa INTO reg;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Pessoa: %', reg.nome;
    END LOOP;

    CLOSE cur_pessoa; -- 4. encerramento
END;
$$ LANGUAGE plpgsql;
```

Etapas explícitas:

1. declaração do cursor;
2. abertura (OPEN);
3. leitura (FETCH);
4. encerramento (CLOSE).

2.8. Uso de cursores em triggers (cuidados)

Embora seja possível usar cursores em funções de trigger, **isso deve ser feito com cautela**, pois:

- triggers são executados automaticamente;
- podem ser disparados muitas vezes;
- cursores aumentam o custo de execução.

Boa prática:

- evitar cursores explícitos em triggers;
- preferir operações simples e diretas;
- usar cursores apenas quando estritamente necessário.

2.9. Boas práticas com triggers

- manter triggers simples e objetivos;
- documentar claramente sua finalidade;
- evitar lógica pesada dentro de triggers;
- testar impactos de desempenho;
- lembrar que triggers podem afetar todas as aplicações.

iii. Triggers Avançados: INSTEAD OF e TRUNCATE

3.1. Limitações dos triggers tradicionais

Triggers BEFORE e AFTER:

- são associados a **tabelas físicas**;
- funcionam com INSERT, UPDATE e DELETE;
- permitem validação, auditoria e ajuste de dados.

Entretanto, existem situações em que:

- não há uma tabela física diretamente envolvida;
- a operação padrão precisa ser **substituída**, e não apenas complementada;
- comandos como TRUNCATE não disparam triggers por linha.

Para esses casos, o PostgreSQL oferece **mecanismos específicos**.

3.2. Triggers INSTEAD OF

2.1 O que é um trigger INSTEAD OF?

Um trigger INSTEAD OF é um trigger que:

- é definido **sobre uma view**, e não sobre uma tabela;
- substitui completamente a operação original;
- é executado **no lugar** (INSTEAD OF) do INSERT, UPDATE ou DELETE.

Definição conceitual:

Um trigger INSTEAD OF executa uma lógica alternativa no lugar da operação solicitada sobre uma view.

3.2.2 Por que INSTEAD OF é necessário?

Views:

- não armazenam dados;
- muitas vezes não são atualizáveis automaticamente;
- não sabem como propagar alterações para múltiplas tabelas.

O trigger **INSTEAD OF** permite:

- interceptar a operação na view;
- definir manualmente como os dados devem ser tratados;
- tornar uma view **atualizável de forma controlada**.

3.3 Exemplo Prático — INSTEAD OF INSERT em View

Cenário: Criar uma view que exibe pessoas e permitir inserções nessa view, propagando os dados para a tabela pessoa.

View

```
CREATE OR REPLACE VIEW vw_pessoa AS
SELECT id_pessoa, nome, nascimento
FROM pessoa;
```

Função de trigger

```
CREATE OR REPLACE FUNCTION inserir_pessoa_view()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO pessoa (id_pessoa, nome, nascimento)
    VALUES (NEW.id_pessoa, NEW.nome, NEW.nascimento);

    RAISE NOTICE 'ID:%, Nome:%, Nascimento:%', NEW.id_pessoa, NEW.nome,
    NEW.nascimento;
    RETURN NULL;
END;
$$;
```

Trigger INSTEAD OF

```
CREATE OR REPLACE TRIGGER trg_inserir_pessoa_view
INSTEAD OF INSERT ON vw_pessoa
FOR EACH ROW
EXECUTE FUNCTION inserir_pessoa_view();
```

Uso

```
INSERT INTO vw_pessoa (id_pessoa, nome, nascimento)
VALUES (100002, 'Maria Novo', '1995-05-20');
```

Nesse caso:

- o INSERT não ocorre na view;
- o trigger intercepta a operação;
- os dados são inseridos na tabela base.

Observações importantes sobre INSTEAD OF

- só pode ser usado em **views**;
- não existe BEFORE ou AFTER em views;
- a função normalmente retorna NULL;
- exige atenção para garantir consistência dos dados.

3.4. Trigger para o evento TRUNCATE

3.4.1 Particularidades do TRUNCATE

O comando TRUNCATE:

- remove todos os registros de uma tabela;
- é um comando DDL, não DML;
- é muito mais rápido que DELETE;
- **não dispara triggers por linha.**

Por isso, ele exige um tipo especial de trigger.

3.4.2 Trigger AFTER TRUNCATE

No PostgreSQL, é possível criar triggers para o evento TRUNCATE, mas com as seguintes características:

- só podem ser do tipo **AFTER**;
- são executados **uma única vez por comando**;
- não possuem acesso a OLD ou NEW.

3.5. Exemplo prático — Trigger AFTER TRUNCATE

Cenário: Registrar quando a tabela carro for truncada.

Tabela de log

```
DROP TABLE IF EXISTS log_truncate;
CREATE TABLE log_truncate (
    tabela VARCHAR(50),
    data_evento TIMESTAMP
);
```

Função de trigger

```
CREATE OR REPLACE FUNCTION log_truncate_carro()
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO log_truncate
    VALUES ('carro', CURRENT_TIMESTAMP);

    RETURN NULL;
END;
$$;
```

Trigger AFTER TRUNCATE

```
CREATE OR REPLACE TRIGGER trg_log_truncate_carro
AFTER TRUNCATE ON carro
FOR EACH STATEMENT
EXECUTE FUNCTION log_truncate_carro();
```

Uso

```
TRUNCATE TABLE carro;
```

Resultado:

- todos os registros são removidos;
- o evento é registrado na tabela log_truncate.

```
SELECT * from log_truncate;
```

tabela	data_evento
character varying (50)	timestamp without time zone
carro	2026-01-31 19:09:46.417023

Limitações dos triggers de TRUNCATE

- não é possível impedir o TRUNCATE com trigger;
- não há acesso aos dados removidos;
- uso típico é **auditoria e monitoramento**.

3.6. Comparação geral dos tipos de trigger

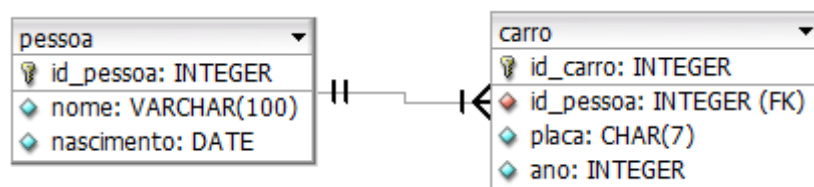
Tipo de Trigger	Onde atua	Quando executa	Uso típico
BEFORE	Tabela	Antes da operação	Validação
AFTER	Tabela	Após a operação	Auditoria
INSTEAD OF	View	No lugar da operação	Atualização de views
AFTER TRUNCATE	Tabela	Após o truncate	Log e controle

3.7. Boas práticas em triggers avançados

- usar INSTEAD OF apenas quando necessário;
- documentar claramente a lógica do trigger;
- evitar lógica complexa em triggers de TRUNCATE;
- lembrar que triggers são executados automaticamente;
- testar impactos em cenários reais.

Exercícios

Para a correta execução dos exercícios, utilize os comandos a seguir para criar as tabelas e carregar os registros a partir de arquivos CSV.



Cada arquivo contém 100 mil registros.

Antes de executar os comandos COPY, é necessário **ajustar o caminho dos arquivos** de acordo com a localização dos CSVs no seu computador.

<pre> DROP TABLE IF EXISTS carro, pessoa CASCADE; CREATE TABLE IF NOT EXISTS pessoa (id_pessoa INTEGER PRIMARY KEY, nome VARCHAR(100) NOT NULL, nascimento DATE); CREATE TABLE IF NOT EXISTS carro (id_carro INTEGER PRIMARY KEY, placa CHAR(7) NOT NULL UNIQUE, ano INTEGER, id_pessoa INTEGER NOT NULL, FOREIGN KEY (id_pessoa) REFERENCES pessoa(id_pessoa) ON DELETE CASCADE); </pre>	<pre> COPY pessoa (id_pessoa, nome, nascimento) FROM 'C:/caminho/aula3_pessoa.csv' DELIMITER ',' CSV HEADER; COPY carro (id_carro, placa, ano, id_pessoa) FROM 'C:/caminho/aula3_carro.csv' DELIMITER ',' CSV HEADER; </pre>
---	--

Exercício 1 — Trigger de validação (BEFORE INSERT)

Crie um trigger que impeça a inserção de um carro com ano maior que o ano atual.

Resposta:

Função de trigger

```
CREATE OR REPLACE FUNCTION fn_validar_ano_carro()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Comparamos o ano inserido com o ano atual do sistema
    IF NEW.ano > EXTRACT(YEAR FROM CURRENT_DATE) THEN
        RAISE EXCEPTION 'Não é possível cadastrar um carro com ano (%)
superior ao ano atual (%)',
            NEW.ano, EXTRACT(YEAR FROM CURRENT_DATE);
    END IF;

    -- Se estiver tudo certo, permite a operação
    RETURN NEW;
END;
$$;
```

Trigger associado

```
CREATE OR REPLACE TRIGGER trg_impedir_ano_futuro
BEFORE INSERT OR UPDATE ON carro
FOR EACH ROW
EXECUTE FUNCTION fn_validar_ano_carro();
```

Uso:

```
INSERT INTO CARRO (id_carro, placa, ano, id_pessoa)
VALUES (1, 'AAA1B23', 2030, 1);
```

Exercício 2 — Trigger de auditoria (AFTER UPDATE)

Crie um trigger que registre em uma tabela de auditoria toda alteração de ano na tabela carro.

Resposta:

Tabela de auditoria

```
DROP TABLE IF EXISTS carro_auditoria;
CREATE TABLE carro_auditoria (
    id_carro INTEGER,
    ano_antigo INTEGER,
    ano_novo INTEGER,
```

```
        data_alteracao TIMESTAMP
    );
```

Função de trigger

```
CREATE OR REPLACE FUNCTION auditar_ano_carro()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO carro_auditoria
    VALUES (
        OLD.id_carro,
        OLD.ano,
        NEW.ano,
        CURRENT_TIMESTAMP
    );

    RETURN NEW;
END;
$$;
```

Trigger associado

```
CREATE OR REPLACE TRIGGER trg_auditar_ano_carro
AFTER UPDATE OF ano ON carro
FOR EACH ROW
EXECUTE FUNCTION auditar_ano_carro();
```

Uso:

```
UPDATE CARRO
SET ano = 2021
WHERE id_carro = 1;
```

Exercício 3 — Uso de OLD em DELETE

Crie um trigger que registre a placa do carro removido sempre que um DELETE for executado na tabela carro.

Resposta:

Tabela de auditoria

```
DROP TABLE IF EXISTS carro_removido;
CREATE TABLE carro_removido (
    id_carro INTEGER,
    placa CHAR(7),
    ano INTEGER,
    data_remocao TIMESTAMP
```

```
);
```

Função de trigger

```
CREATE OR REPLACE FUNCTION log_remocao_carro()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO carro_removido
    VALUES (OLD.id_carro, OLD.placa, OLD.ano, CURRENT_TIMESTAMP);

    RETURN OLD;
END;
$$;
```

Trigger associado

```
CREATE OR REPLACE TRIGGER trg_log_remocao_carro
AFTER DELETE ON carro
FOR EACH ROW
EXECUTE FUNCTION log_remocao_carro();
```

Uso:

```
DELETE FROM carro;
```

Exercício 4 — Trigger com cursor implícito

Crie um bloco PL/pgSQL que percorra todas os carros e exiba sua placa e ano utilizando um cursor implícito.

Resposta:

```
DO $$
DECLARE
    reg RECORD;
BEGIN
    FOR reg IN
        SELECT placa, ano FROM carro
    LOOP
        RAISE NOTICE '% - %', reg.placa, reg.ano;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Exercício 5 — Trigger INSTEAD OF INSERT em view

Crie uma view baseada na tabela carro e permita inserções nessa view utilizando um trigger INSTEAD OF.

Resposta:

View

```
CREATE OR REPLACE VIEW vw_carro AS
SELECT id_carro, placa, ano, id_pessoa
FROM carro;
```

Função de trigger

```
CREATE OR REPLACE FUNCTION inserir_carro_view()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO carro (id_carro, placa, ano, id_pessoa)
    VALUES (NEW.id_carro, NEW.placa, NEW.ano, NEW.id_pessoa);

    RETURN NULL;
END;
$$;
```

Trigger

```
CREATE OR REPLACE TRIGGER trg_insert_vw_carro
INSTEAD OF INSERT ON vw_carro
FOR EACH ROW
EXECUTE FUNCTION inserir_carro_view();
```

Uso

```
INSERT INTO vw_carro (id_carro, placa, ano, id_pessoa)
VALUES (10, 'ABC1D23', '1999', 5);
```

Exercício 6 — Questão conceitual (NEW × OLD)

Explique a diferença entre NEW e OLD em triggers.

Resposta:

- NEW representa a linha nova, usada em INSERT e UPDATE;
- OLD representa a linha antiga, usada em UPDATE e DELETE;
- não estão disponíveis em triggers de TRUNCATE.

Exercício 7 — Questão conceitual (Cursors)

Explique o que é um cursor e a diferença entre cursor implícito e explícito.

Resposta:

- Um cursor permite percorrer um conjunto de resultados **linha a linha**;

- cursor implícito é usado automaticamente em FOR ... IN SELECT;
- cursor explícito exige DECLARE, OPEN, FETCH e CLOSE;
- cursores explícitos oferecem mais controle, mas são menos usados.